

Albina Aksolakov - Cosima Baumeier - David Benkel - Romina Fernandez

# HANABI SCORE PREDICTION

Applied Machine Learning  
[open-campus.sh](https://open-campus.sh)



# INTRODUCTION

## GAME:

- Cards 1–5 in 5 different colors → must be played in ascending order
- Catch: you only see your teammates' cards, never your own
- Hints are limited and restricted strictly to specific numbers or colors

## PROJECT QUESTION:

*Can we accurately predict the final score before the very first move is made, purely based on the starting cards?*

- Extracting derived features from the starting hands → Multi-Layer Perceptron → predicting final score from 0 to 25

# LITERATURE REVIEW

- How to featurize abstract board states into structured numerical vectors
- Regression pipeline is the ideal tool → Linear Regression model as our baseline
- Literature emphasizes that human communication and in-game strategy are the true drivers of success - ?

# DATASET CHARACTERISTICS

- Game log data
- Full move-by-move trace until final score
- 2,040 observations (1,810 used)

## **Target variable**

- Final score (0-25)

## **Raw encoding**

- Hand encoding: 5x5x2

## **Derived Features**

- Total count of each number
- Total count of each color
- Total hand values
- Average hand values
- Unique card count
- Avg. max. possible progression

# BASELINE MODEL

- Model type: Linear regression
- Evaluation Metric: RMSE
- Data Split: 80/20 train-test split

|                | Test split | 5-fold CV        |
|----------------|------------|------------------|
| MSE            | 32.70      | $33.01 \pm 0.55$ |
| RMSE           | 5.72       | $5.75 \pm 0.05$  |
| MAE            | 4.95       | $5.00 \pm 0.05$  |
| R <sup>2</sup> | -0.04      | $-0.01 \pm 0.02$ |



# MODEL DEFINITION

```
# Dynamic model definition
class HanabiDynamic(nn.Module):
    def __init__(
        self,
        input_dim,
        num_hidden_layers,
        hidden_size,
        dropout_rate
    ):
        super().__init__()

        layers = []

        current_dim = input_dim

        for _ in range(num_hidden_layers):
            layers.append(
                nn.Linear(current_dim, hidden_size)
            )

            layers.append(nn.ReLU())

            layers.append(
                nn.Dropout(dropout_rate)
            )

            current_dim = hidden_size

        layers.append(
            nn.Linear(current_dim, 1)
        )

        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

## Multilayer Perceptron Neural Network (MLP)

- Dynamic model definition for hyperparameter optimization
- Model structure:
  - Fully connected layers
  - ReLU activation function
  - Dropout regularization



# MODEL DEFINITION

```
model = HanabiDynamic(  
    input_dim=X_train.shape[1],  
    num_hidden_layers=num_hidden_layers,  
    hidden_size=hidden_size,  
    dropout_rate=dropout_rate  
)  
criterion = nn.MSELoss()  
optimizer = torch.optim.Adam(  
    model.parameters(),  
    lr=learning_rate,  
    weight_decay=weight_decay  
)
```

```
# Training  
model.train()  
running_train_loss = 0.0  
  
for X_batch, y_batch in train_loader:  
    optimizer.zero_grad()  
  
    outputs = model(X_batch)  
    loss = criterion(outputs, y_batch)  
  
    loss.backward()  
    optimizer.step()  
  
    running_train_loss += loss.item()  
  
epoch_train_loss = running_train_loss / len(train_loader)  
train_losses.append(epoch_train_loss)
```

## Multilayer Perceptron Neural Network (MLP)

- Optimizer: Adam
- Loss function: MSE Loss
- Learning rate scheduler
  - patience: 3
  - factor: 0.5
- Early stopping
  - patience: 10
  - max epoch: 100

# **HYPERPARAMETER**

## **Search space**

### **Model architecture**

- Number of hidden layers: 2, 3, 4
- Size of hidden layers: 32, 64, 128, 256

### **Regularization**

- Dropout rate: 0.0 - 0.5
- Weight decay:  $1e-6$  -  $1e-2$

### **Optimization**

- Learning rate:  $1e-5$  -  $1e-2$
- batch size: 16, 32, 64, 128

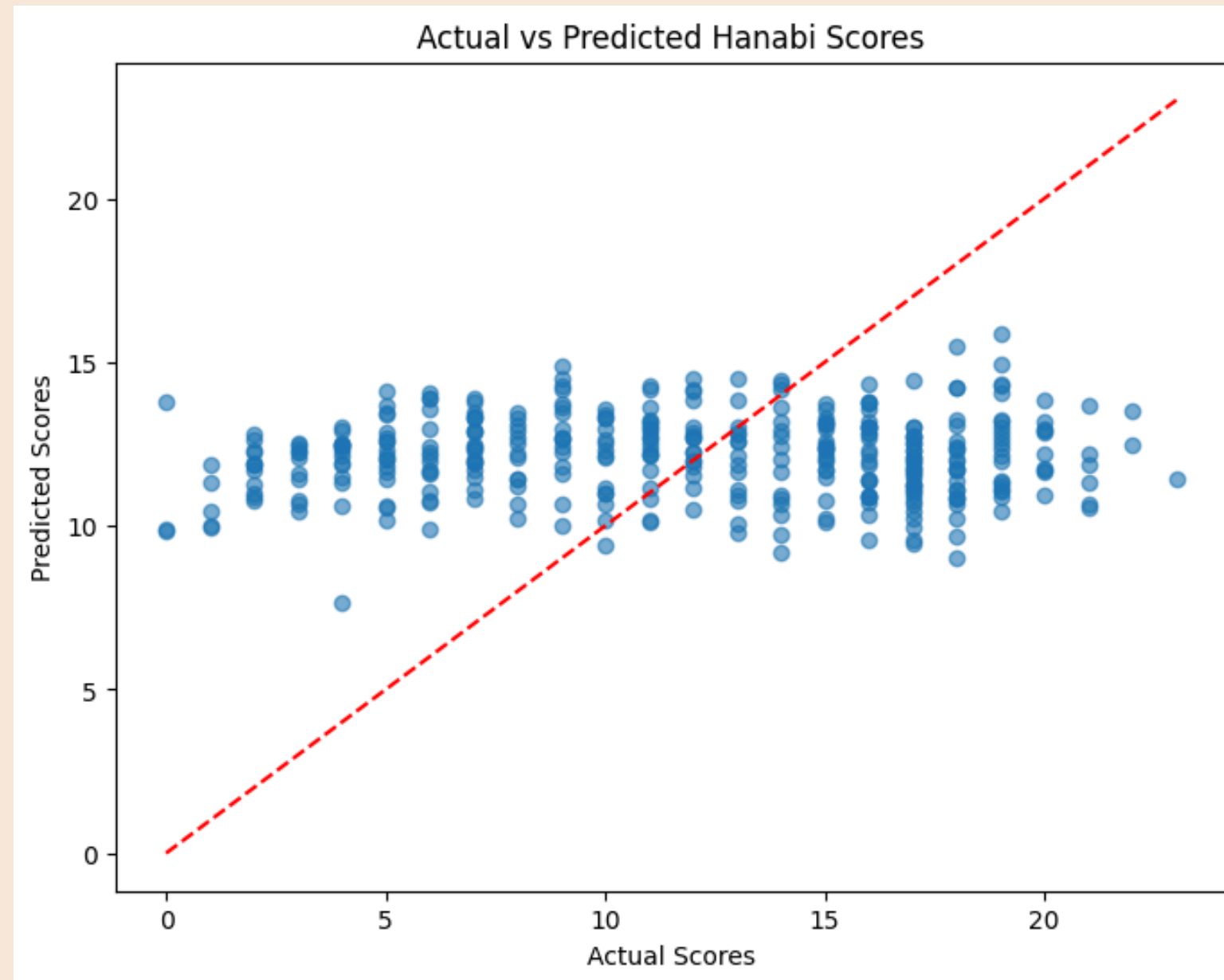
**Optuna**

**500 trials**

**Evaluation metric: MSE**



# EVALUATION



**MSE:** 32.3247

**RMSE:** 5.6855

**MAE:** 4.9420

→ On average, predictions are ~ 4.9 points away from true score

**R<sup>2</sup> Score:** -0.0300

→ The model performs slightly worse than predicting the mean score

# EVALUATION

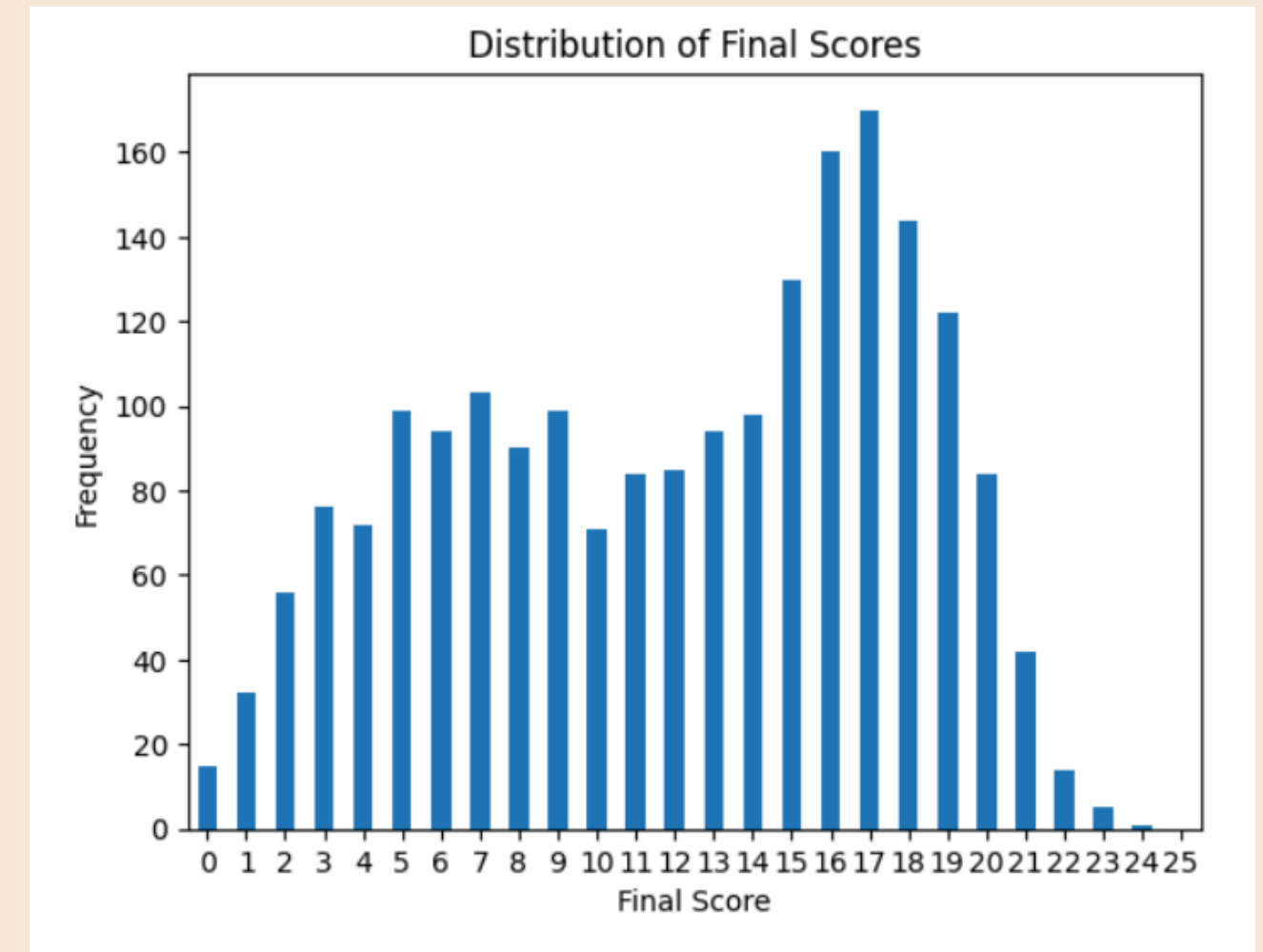
|       | Baseline<br>Linear Regression | Multilayer Perceptron<br>Neural Network |
|-------|-------------------------------|-----------------------------------------|
| MSE   | 32.69                         | 32.32                                   |
| RMSE  | 5.71                          | 5.69                                    |
| MAE   | 4.95                          | 4.94                                    |
| $R^2$ | - 0.04                        | - 0.03                                  |

# DISCUSSION

- Bad performance of the Multilayer Perceptron Neural Network despite feature engineering and hyperparameter optimization → similar to the linear regression baseline
- MAE of almost 5: on average, the prediction is off by five points
- Negative  $R^2$  score: performing worse than predicting the mean
- Underestimating very good games and overestimating very bad games
- → Current feature representation is insufficient to accurately predict final game scores

# CHALLENGES

- Relatively small sample size
- Imbalanced target variable (final score):  
very low and very high scores are rare
- Overrepresentation of some starting hands  
→ we used a random sample of only 3 games  
per starting hands



# CONCLUSION & FUTURE WORK

- Starting hands do not seem to be a good predictor for the final outcome of a Hanabi game  
→ Good for Hanabi fans? ;)
- Future work:
  - Expanded feature engineering; including later stages of the game
  - Collecting more game data for training
  - Testing of alternative architectures or wider hyperparameter search ranges



Albina Aksolakov - Cosima Baumeier - David Benkel - Romina Fernandez

# THANK YOU

QUESTIONS?

